Mapa mental: crear una API REST con ASP.NET Core

Los 10 pasos del manual alrededor de la API. Orden: columna izquierda de arriba abajo (0–4) y luego columna derecha (5–9).

MAPA VISUAL · complemento del manual
Manual_API_ASPNET_Core.pdf

0 · Paso 0 · Requisitos previos — cap. 2

- Visual Studio + carga «ASP.NET and web development»
- .NET 10 LTS → VS 2026 · .NET 8 → soporte hasta nov-2026
- BD creada y nombre exacto del servidor (SSMS)
- Database First (BD ya existe) o Code First

1 · Paso 1 · Crear el proyecto — cap. 3

- Plantilla «ASP.NET Core Web API» (C#)
- Marcar: Use controllers · Enable OpenAPI
- Ya crea: Program.cs, appsettings, Controllers/
- CLI: dotnet new webapi --use-controllers

2 · Paso 2 · Paquetes NuGet — cap. 4

- EntityFrameworkCore.SqlServer (proveedor) + Tools
- Misma versión mayor que .NET (net8 → 8.0.x)
- Clic der. en el PROYECTO → Administrar NuGet
- Compilar: Ctrl+Shift+B (0 errores)

3 · Paso 3 · Modelos — cap. 5

- Models/«Tabla».cs → 1 clase por tabla
- 1 propiedad por columna · si acepta NULL → tipo?
- [Key] si la llave no se llama Id / «Tabla»Id
- Atajo N tablas: Scaffold-DbContext

4 · Paso 4 · DbContext — cap. 6

- Context/AppDbContext.cs
- class AppDbContext : DbContext (hereda)
- Constructor(DbContextOptions<AppDbContext>)
- 1 DbSet<«Tabla»> «Tabla» por cada tabla

API REST

ASP.NET Core Web API + EF Core



HTTP + JSON · SQL generado por EF Core

- Preparación (0–2)
- Capa de datos (3–5)
- Configuración (6, 8)
- Endpoints (7)
- Prueba (9)

Por cada tabla: 1 modelo + 1 DbSet + 1 controlador

5 · Paso 5 · Cadena de conexión — cap. 7

- appsettings.json → sección "ConnectionStrings"
- "«NombreConexion»": "Server=...;Database=..."
- SQL Server local: TrustServerCertificate=True
- Contraseñas → User Secrets, no a git

6 · Paso 6 · Program.cs — cap. 8

- ANTES de Build: AddDbContext(UseSqlServer(...))
- AddCors(«Politica») + WithOrigins(frontend)
- DESPUÉS de Build: UseCors → MapControllers
- .NET 9/10: AddOpenApi + UI de Swagger opcional

7 · Paso 7 · Controladores — cap. 9

- Controllers/«Tabla»Controller.cs (API en blanco)
- [Route("api/[controller]")] + [ApiController]
- private readonly AppDbContext _context (DI)
- GET · GET {id} · POST · PUT · DELETE (async)

8 · Paso 8 · launchSettings.json — cap. 10

- Properties/launchSettings.json
- applicationUrl = URL de la API (va al frontend)
- launchUrl: "swagger" = página al iniciar
- Sin espacio tras «;» · sin comentarios //

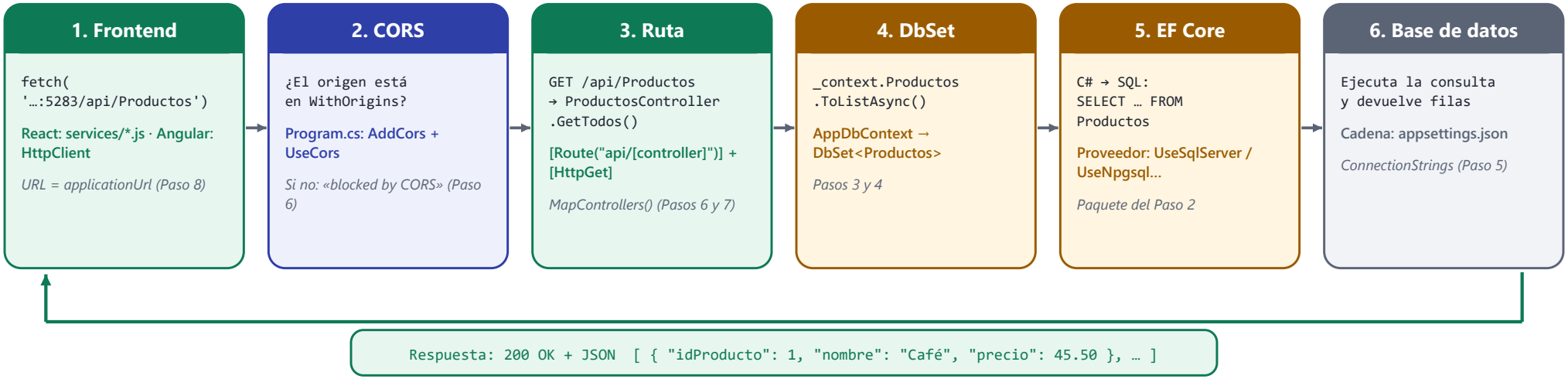
9 · Paso 9 · Ejecutar y probar — cap. 11

- F5 (depurar) · Ctrl+F5 · dotnet run
- Swagger → Try it out → Execute
- .http · Postman · navegador (solo GET)
- Errores frecuentes → tabla 11.3

«Marcador» = valor de tu proyecto (igual que en el manual). cap. N = capítulo del manual con el detalle.

Cómo viaja una petición

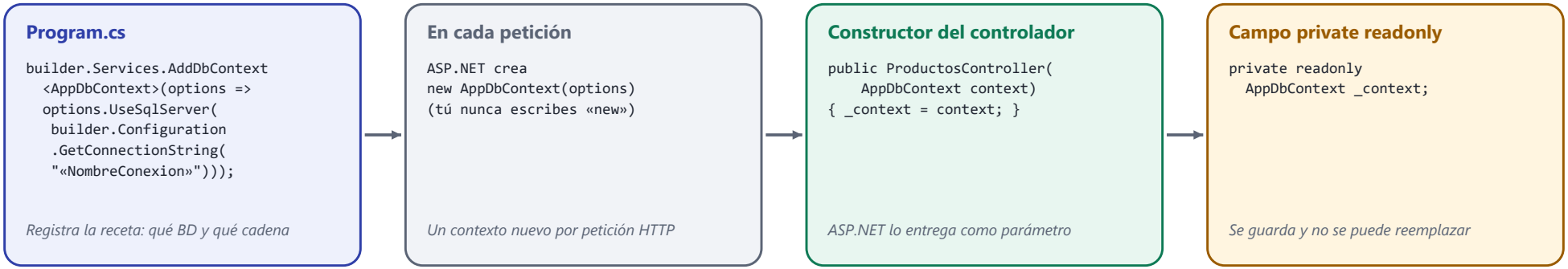
Ejemplo: el frontend pide la lista de productos. Cada caja dice qué archivo participa y en qué paso del manual se configura.



CORS lo aplica el navegador

Solo frenan las páginas web de otro origen (otro puerto o dominio); Postman, apps móviles y otros backends no necesitan CORS. Revisa: origen exacto (sin «/» final), AllowAnyHeader() y AllowAnyMethod(), y UseCors antes de MapControllers (cap. 8.3).

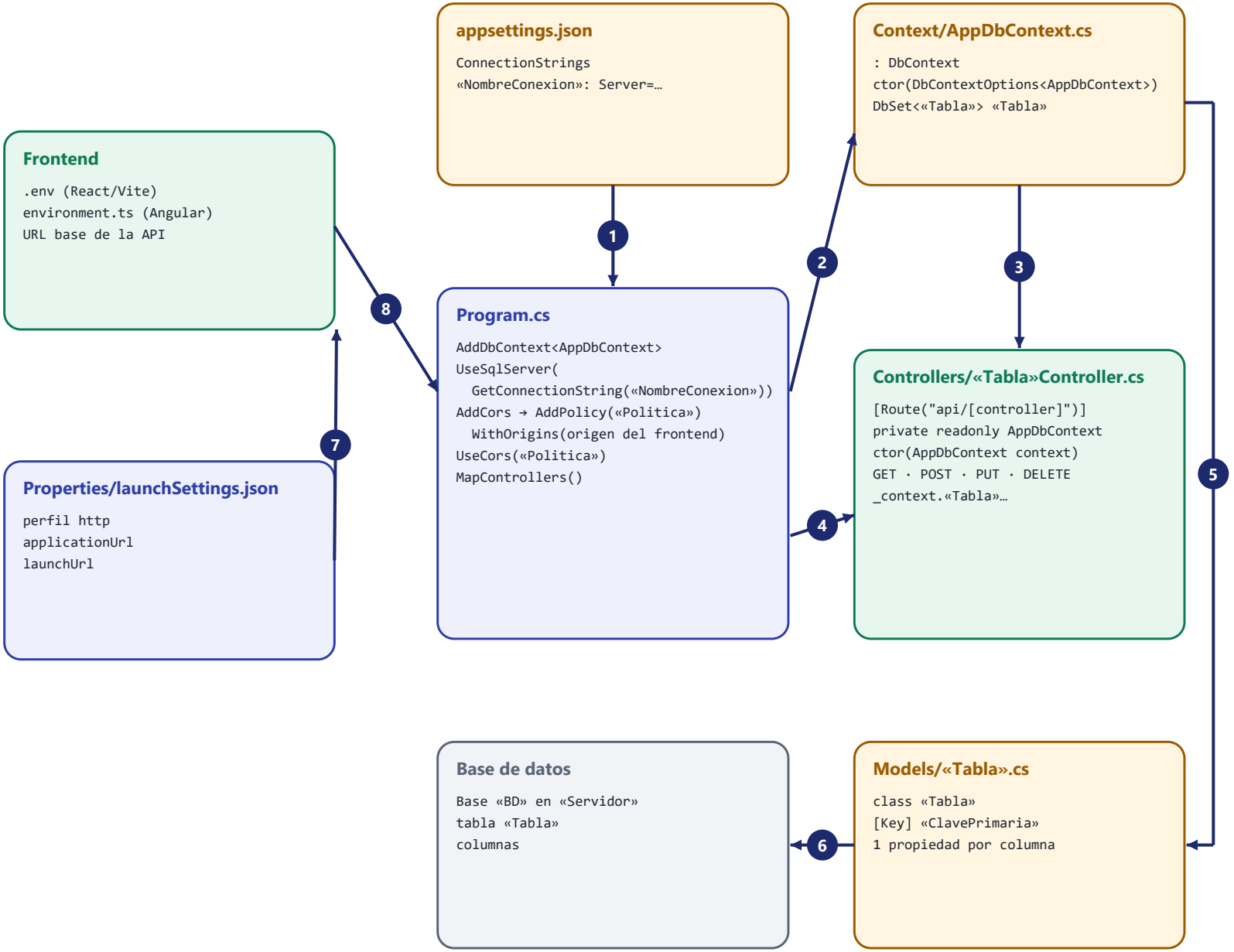
¿Quién crea el DbContext? Inyección de dependencias (cap. 8.2 y 9.3)



«Marcador» = valor de tu proyecto (igual que en el manual). cap. N = capítulo del manual con el detalle.

Qué conecta con qué: archivos y nombres que deben coincidir

Las flechas numeradas son los enlaces que más errores causan cuando un nombre no coincide. La tabla de la derecha los resume.



Deben coincidir

nombres y valores entre archivos

1. **appsettings → Program.cs**
   «NombreConexion» igual en ConnectionStrings y en GetConnectionString("...")
2. **Program.cs → AppDbContext**
   AddDbContext<AppDbContext> entrega las opciones al constructor
3. **AppDbContext → Controlador**
   _context.«Tabla» usa el nombre del DbSet
4. **Program.cs → Controlador**
   La DI entrega AppDbContext al constructor (sin AddDbContext: error)
5. **DbSet → Modelo**
   DbSet<«Tabla»> apunta a la clase de Models/
6. **Modelo → Tabla BD**
   Nombre del DbSet (o [Table]) = nombre real de la tabla; [Key] = llave
7. **launchSettings → Frontend**
   applicationUrl (puerto) = URL base en .env / environment.ts
8. **Frontend → CORS**
   Origen del frontend (http://localhost:5173 / 4200) en WithOrigins

+ **Dentro de Program.cs**
  «Politica» igual en AddPolicy("...") y UseCors("...")

«Marcador» = valor de tu proyecto (igual que en el manual). cap. N = capítulo del manual con el detalle.

Qué cambia si… · Guía rápida del CRUD

Izquierda: qué pasos tocar si cambias de base de datos, frontend, versión o herramienta (cap. 12). Derecha: los 5 endpoints de cada tabla (cap. 9).

Paso	Otra base de datos	Otro frontend	Otra versión de .NET	VS Code / terminal
1 Proyecto	No cambia	No cambia	Elegir .NET; VS 2026 para .NET 10	dotnet new webapi --use-controllers
2 NuGet	Otro proveedor (Npgsql, Pomelo…)	No cambia	Versión EF = versión .NET	dotnet add package
3 Modelos	Tipos equivalentes	No cambia	No cambia	Crear .cs a mano
4 DbContext	No cambia	No cambia	No cambia	No cambia
5 Conexión	Otro formato de cadena	No cambia	No cambia	No cambia
6 Program.cs	UseSqlServer → UseNpgsql…	Solo el origen en CORS	Swagger: AddOpenApi (9/10)	No cambia
7 Controladores	No cambia	No cambia	No cambia	Crear .cs a mano
8 launchSettings	No cambia	No cambia	launchUrl de Swagger/Scalar	dotnet run --launch-profile
9 Probar	No cambia	Probar desde el nuevo front	No cambia	dotnet run

No cambia · Cambia poco · Cambia (revisar)

Los 5 endpoints de cada «Tabla»Controller

GET `/api/«Tabla»`
`GetTodos()` → `ToListAsync()`
Responde: 200 lista

GET `/api/«Tabla»/{id}`
`GetPorId(id)` → `FindAsync(id)`
Responde: 200 · 404

POST `/api/«Tabla»`
`Crear(entidad)` → `Add + SaveChangesAsync`
Responde: 201 · 400

PUT `/api/«Tabla»/{id}`
`Actualizar(id, datos)` → `SetValues + SaveChangesAsync`
Responde: 204 · 400 · 404

DELETE `/api/«Tabla»/{id}`
`Eliminar(id)` → `Remove + SaveChangesAsync`
Responde: 204 · 404

Códigos que verás

200 OK con datos · 201 Creado · 204 Hecho, sin datos
400 Datos inválidos · 404 No existe · 500 Falla servidor/BD

Por cada tabla nueva repite esta secuencia

1. **`Models/«Tabla».cs`** — 1 prop. por columna (Paso 3)
2. **`DbSet<«Tabla»> «Tabla»`** — en AppDbContext (Paso 4)
3. **`«Tabla»Controller.cs`** — 5 métodos CRUD (Paso 7)
4. **`Compilar · Ctrl+Shift+B`** — 0 errores (Paso 2)
5. **`Probar en Swagger`** — los 5 endpoints (Paso 9)

«Marcador» = valor de tu proyecto (igual que en el manual). cap. N = capítulo del manual con el detalle.